

Symbiosis Institute of Technology, Nagpur

DEPARTMENT OF COMPUTER SCIENCE • JAVASCRIPT PROGRAMMING LAB

Student: Tanmay Sankulwar

PRN: 24070521058

Course: Semester V (B.Tech)

EXPERIMENT 9

Student Name: Tanmay Sankulwar

PRN: 24070521058

File Path: EXP9/index.html , EXP9/Casestudy/

Experiment Title

Theme-Responsive Student Admission Portal & Seminar Schedule Planner using Web Storage API (`localStorage` & `sessionStorage`)

Objective

1. To understand and apply the HTML5 Web Storage API (`localStorage` and `sessionStorage`) for client-side state persistence without server roundtrips.
2. To implement a persistent Dark and Light theme toggle that stores user preferences in `localStorage` and eliminates Flash of Unstyled Content (FOUC).
3. To develop an auto-saving draft form feature using `sessionStorage` that caches inputs on keystroke and restores data upon accidental page reloads.
4. To build an academic Seminar Schedule Planner using complex HTML table layouts (`rowspan` , `colspan`), event-driven topic selection, and session tracking.
5. To implement state cleanup mechanisms using `sessionStorage.removeItem()` , `sessionStorage.clear()` , and `localStorage.clear()` .

Software / Tools Required

1. **Operating System:** Windows 10 / 11
2. **Development Environment:** Visual Studio Code / Antigravity IDE
3. **Web Browser:** Google Chrome / Modern Web Browser
4. **Languages & Technologies:** HTML5, CSS3 (CSS Variables / Custom Properties, Flexbox, Responsive Design), Modern JavaScript (ES6+)

Program / Task

Task 9.a — Student Admission Form with Dynamic Theme & Draft Persistence

File: EXP9/index.html

The application provides a modern, responsive **Student Admission Form** featuring:

- **Theme Switching:** An interactive **Switch Theme** button toggling between light and dark visual aesthetics using CSS custom properties (`var(--bg-color)` , `var(--card-bg)` , `var(--text-color)`). The selected theme is stored in `localStorage` so it persists across browser restarts.
- **Immediate FOUC Mitigation:** An inline script inside the `<head>` checks `localStorage` prior to DOM rendering, ensuring smooth theme hydration without screen flickering.
- **Form Draft Auto-Save:** Every keystroke across form inputs (Full Name, Email Address, Phone Number, and Desired Course) triggers an `input` event that writes the current value to `sessionStorage`.
- **Form State Restoration:** On `DOMContentLoaded` , existing session keys are inspected to populate all previously entered values automatically.
- **Submission & Cleanup:** Upon form submission (`submit`), the application prevents default reloading, triggers a submission confirmation alert, purges all stored draft session keys, and invokes `form.reset()` .

Source Code

Main Application — Student Admission Form (`EXP9/index.html`)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Admission Form</title>

  <style>
    :root {
      --bg-color: #eef3f8;
      --card-bg: #ffffff;
      --text-color: #1f2937;
      --border-color: #d6dee8;
      --input-bg: #f8f9fc;
      --primary-color: #2563eb;
      --primary-hover: #1d4ed8;
      --toggle-bg: #e2e8f0;
      --toggle-text: #1e293b;
      --footer-text: #64748b;
      --shadow: rgba(15, 23, 42, 0.10);
    }

    body.dark-theme {
      --bg-color: #0f172a;
      --card-bg: #1e293b;
      --text-color: #f1f5f9;
      --border-color: #475569;
      --input-bg: #273449;
      --primary-color: #3b82f6;
      --primary-hover: #2563eb;
      --toggle-bg: #334155;
      --toggle-text: #f8f9fc;
      --footer-text: #94a3b8;
      --shadow: rgba(0, 0, 0, 0.30);
    }

    * {
      box-sizing: border-box;
    }

    body {
      margin: 0;
      min-height: 100vh;
      padding: 30px 20px;
      font-family: "Segoe UI", Arial, sans-serif;
      background: var(--bg-color);
      color: var(--text-color);
      display: flex;
```

```
    flex-direction: column;
    align-items: center;
    transition: 0.3s ease;
}

.header-actions {
    width: 100%;
    max-width: 620px;
    display: flex;
    justify-content: flex-end;
    margin-bottom: 15px;
}

.theme-btn {
    padding: 9px 18px;
    border: 1px solid var(--border-color);
    border-radius: 8px;
    background: var(--toggle-bg);
    color: var(--toggle-text);
    font-size: 14px;
    font-weight: 600;
    cursor: pointer;
    transition: 0.3s ease;
}

.theme-btn:hover {
    transform: translateY(-1px);
    opacity: 0.9;
}

.form-card {
    width: 100%;
    max-width: 620px;
    padding: 35px;
    background: var(--card-bg);
    border: 1px solid var(--border-color);
    border-radius: 14px;
    box-shadow: 0 12px 30px var(--shadow);
    transition: 0.3s ease;
}

h2 {
    margin: 0 0 28px;
    text-align: center;
    font-size: 28px;
    color: var(--text-color);
}

.form-group {
    margin-bottom: 20px;
}

label {
    display: block;
    margin-bottom: 7px;
    font-size: 14px;
    font-weight: 600;
}

input,
select {
    width: 100%;
    padding: 12px 13px;
    border: 1px solid var(--border-color);
    border-radius: 8px;
    background: var(--input-bg);
    color: var(--text-color);
    font-size: 15px;
    transition: 0.25s ease;
}

input::placeholder {
```

```
    color: #94a3b8;
  }

input:focus,
select:focus {
  outline: none;
  border-color: var(--primary-color);
  box-shadow: 0 0 0 3px rgba(37, 99, 235, 0.12);
}

.submit-btn {
  width: 100%;
  margin-top: 8px;
  padding: 13px;
  border: none;
  border-radius: 8px;
  background: var(--primary-color);
  color: white;
  font-size: 16px;
  font-weight: 600;
  cursor: pointer;
  transition: 0.3s ease;
}

.submit-btn:hover {
  background: var(--primary-hover);
  transform: translateY(-1px);
}

footer {
  margin-top: 25px;
  text-align: center;
  color: var(--footer-text);
  font-size: 14px;
  font-weight: 500;
}
</style>

<script>
const localTheme = localStorage.getItem("theme");
const sessionTheme = sessionStorage.getItem("theme");

if (localTheme === "dark" || sessionTheme === "dark") {
  document.documentElement.classList.add("dark-theme");
}
</script>
</head>

<body>

<div class="header-actions">
  <button class="theme-btn" id="theme-toggle">Switch Theme</button>
</div>

<div class="form-card">
  <h2>Student Admission Form</h2>

  <form id="admissionForm">

    <div class="form-group">
      <label for="fullName">Full Name</label>
      <input
        type="text"
        id="fullName"
        placeholder="Enter full name"
        required>
    </div>

    <div class="form-group">
      <label for="emailAddress">Email Address</label>
      <input
        type="email"

```

```
        id="emailAddress"
        placeholder="name@example.com"
        required>
</div>

<div class="form-group">
  <label for="phoneNumber">Phone Number</label>
  <input
    type="tel"
    id="phoneNumber"
    placeholder="Enter mobile number"
    required>
</div>

<div class="form-group">
  <label for="courseChoice">Desired Course</label>

  <select id="courseChoice" required>
    <option value="">-- Select a Course --</option>
    <option value="Computer Science">
      Computer Science & Engineering
    </option>
    <option value="Information Technology">
      Information Technology
    </option>
    <option value="Data Science">
      Data Science
    </option>
    <option value="Cyber Security">
      Cyber Security
    </option>
  </select>
</div>

<button type="submit" class="submit-btn">
  Submit Application
</button>

</form>
</div>

<footer>
  Name: Tanmay Sankulwar | PRN: 24070521058
</footer>

<script>
  if (document.documentElement.classList.contains("dark-theme")) {
    document.body.classList.add("dark-theme");
    document.documentElement.classList.remove("dark-theme");
  }

  const toggleBtn = document.getElementById("theme-toggle");

  toggleBtn.addEventListener("click", function () {
    const isDarkNow =
      document.body.classList.toggle("dark-theme");

    if (isDarkNow) {
      localStorage.setItem("theme", "dark");
      sessionStorage.removeItem("theme");
    } else {
      sessionStorage.setItem("theme", "light");
      localStorage.removeItem("theme");
    }
  });

  const form = document.getElementById("admissionForm");

  const fields = [
    "fullName",
    "emailAddress",
    "phoneNumber",
```

```
"courseChoice"
];

window.addEventListener("DOMContentLoaded", function () {
  fields.forEach(function (fieldId) {
    const savedValue =
      sessionStorage.getItem(fieldId);

    if (savedValue) {
      document.getElementById(fieldId).value =
        savedValue;
    }
  });
});

fields.forEach(function (fieldId) {
  const inputElement =
    document.getElementById(fieldId);

  inputElement.addEventListener("input", function () {
    sessionStorage.setItem(
      fieldId,
      inputElement.value
    );
  });
});

form.addEventListener("submit", function (e) {
  e.preventDefault();

  alert("Application Form Submitted Successfully!");

  fields.forEach(function (fieldId) {
    sessionStorage.removeItem(fieldId);
  });

  form.reset();
});
</script>

</body>
</html>
```

Explanation / Concepts

1. **localStorage vs sessionStorage** : - **localStorage** : Persists data across browser sessions and tab closures indefinitely until programmatically cleared (`localStorage.clear()` or `.removeItem()`). It is ideal for persistent user preferences such as theme selection (`theme: "dark"`). - **sessionStorage** : Scoped strictly to the lifetime of the specific browser tab or window. Closing the tab discards the data. It is optimal for temporary session states, such as multi-step form drafts and unsaved user inputs.
2. **Flash of Unstyled Content (FOUC) Prevention** : - Standard stylesheet-driven dark themes can momentarily flash white while scripts load at the bottom of the body. - By embedding a lightweight script directly inside the document `<head>`, the stored theme key is verified and applied before the browser engine paints the initial frame.
3. **Real-Time Input Buffering (`input` event)** : - An array containing all input identifiers `["fullName", "emailAddress", "phoneNumber", "courseChoice"]` is iterated using `forEach()` . - Each input element attaches an `input` event listener, synchronizing user keystrokes straight to session memory using `sessionStorage.setItem(fieldId, value)` .
4. **Lifecycle Rehydration (`DOMContentLoaded`)** : - When the page loads or refreshes, `sessionStorage.getItem(fieldId)` checks for existing draft data and populates field values, ensuring complete

resilience against accidental page reload.

5. **Form Cleanup & Reset:** - Upon successful form submission, `sessionStorage.removeItem(fieldId)` purges draft keys so subsequent visits begin with a fresh, clean form.

Output

1. **Default State (Light Theme):** A centered card on a light blue-grey background (`#eef3f8`) with form controls for Full Name, Email Address, Phone Number, and Desired Course.
2. **Theme Toggling:** Clicking **Switch Theme** toggles `.dark-theme`, transforming the interface into a sleek dark palette (`#0f172a` body, `#1e293b` card) that remains active even after refreshing or reopening the browser.
3. **Session Auto-Save:** Entering data into the input fields caches values in `sessionStorage`. Reloading the page immediately restores the typed information.
4. **Form Submission:** Clicking **Submit Application** validates required fields, triggers an alert popup `"Application Form Submitted Successfully!"`, clears cached session keys, and resets the form.

Screenshot

File C:/Users/sanku/OneDrive/Desktop/24070521058_Javascript/EXP9/index.html

Switch Theme

Student Admission Form

Full Name
Enter full name

Email Address
name@example.com

Phone Number
Enter mobile number

Desired Course
-- Select a Course --

Submit Application

Name: Tanmay Sankulwar | PRN: 24070521058

Student Admission Form Output

Case Study

Case Study — Seminar Schedule Planner with Dynamic Storage Management

File: `EXP9/Casestudy/index.html`

The case study provides an academic timetable application illustrating structured HTML table spanning alongside Web Storage management.

Key Features & Architecture:

- **Complex Table Hierarchy:** Uses `rowspan` and `colspan` across columns (`Day` , `Schedule: Begin/End` , and `Topic`) to represent multiday seminar agendas (XML, DTD, Relax NG, XPath, XSL Transformations, and XSL Formatting Objects).
- **Interactive Topic Tracking:** Attaches click listeners to all `.topic` table cells via `querySelectorAll()` . Clicking any topic extracts the `data-topic` attribute, notifies the user via alert dialog, and saves the value to `sessionStorage.setItem("lastTopic", selectedTopic)` .
- **Persistent Status Banner:** An informational panel `#last-topic` displays the most recently selected seminar session, rehydrating its value upon page reload via `sessionStorage.getItem("lastTopic")` .
- **Theme Persistence:** Stores active dark/light preference in `localStorage` under the key `"theme"` .
- **Storage Reset Utility:** The **Clear Data** button executes both `localStorage.clear()` and `sessionStorage.clear()` , resetting the banner state and presenting an alert confirmation.

```
// Excerpt from EXP9/Casestudy/index.html
const themeButton = document.getElementById("theme-toggle");
const clearButton = document.getElementById("clear-data");
const topics = document.querySelectorAll(".topic");
const lastTopic = document.getElementById("last-topic");

// Dark Theme toggle using LocalStorage
themeButton.addEventListener("click", function () {
  const isDark = document.body.classList.toggle("dark-theme");
  if (isDark) {
    localStorage.setItem("theme", "dark");
  } else {
    localStorage.setItem("theme", "light");
  }
});

// Interactive topic selection using sessionStorage
topics.forEach(function (topic) {
  topic.addEventListener("click", function () {
    const selectedTopic = topic.getAttribute("data-topic");
    sessionStorage.setItem("lastTopic", selectedTopic);
    alert("Selected Topic: " + selectedTopic);
    lastTopic.textContent = "Last Selected Topic: " + selectedTopic;
  });
});

// Rehydrate state on page Load
window.addEventListener("DOMContentLoaded", function () {
  const savedTopic = sessionStorage.getItem("lastTopic");
  if (savedTopic) {
    lastTopic.textContent = "Last Selected Topic: " + savedTopic;
  }
});

// Clear all Web Storage data
clearButton.addEventListener("click", function () {
  localStorage.clear();
  sessionStorage.clear();
  lastTopic.textContent = "All saved data has been cleared.";
  alert("Local Storage and Session Storage cleared!");
});
```

Case Study Screenshot

Seminar Schedule Planner

Switch Theme

Clear Data

Day	Schedule		Topic
	Begin	End	
Monday	8:00 a.m.	5:00 p.m.	Introduction to XML
			Validity: DTD and Relax NG
Tuesday	8:00 a.m.	11:00 a.m.	XPath
	11:00 a.m.	2:00 p.m.	XSL Transformations
	2:00 p.m.	5:00 p.m.	XSL Transformations
Wednesday	8:00 a.m.	12:00 p.m.	XSL Formatting Objects

Click on any topic to view its schedule.

Seminar Schedule Planner | JavaScript Practical

Seminar Schedule Planner Output

Result / Conclusion

Experiment 9 was successfully implemented and validated. The practical verified the distinction and practical usage of the HTML5 Web Storage API: employing `localStorage` for long-term user settings (theme preference) and `sessionStorage` for temporary transient state (form draft buffering and dynamic schedule selection). The implementations guaranteed seamless user experience through FOUC prevention and resilient client-side state rehydration.